

```
#include<stdio.h>
int *p;
int main () {
    *p=10;
    printf ("The value of p =%d", *p);
    return 0;
}
```

If you compile and run that, on most Linux systems, you will get a *Segmentation fault (core dumped)* error. How is this error output on screen?

In all three cases, some form of event notification (signal) to a process took place. In the first case, *kill* requested the OS to send a notification, denoted by the number 9, to a process. In the second, the user's *CTRL-C* terminated the process. In the third case, a memory access violation occurred. Basically, exceptional conditions are reported by *signals*, and these are some examples of such conditions. Other general examples can be the expiration of a timer, unexpected errors while executing a system call, or termination of a child process. Standard signals are defined for these types of events, like *SIGSEGV*, *SIGINT* and *SIGUSR1*, etc. Refer to the manual page (*man 7 signal*) for a list of signals.

When a process receives a signal, the action is performed by a *signal handler* function. Such functions are automatically linked to your code for actions in response to OS signals—such as to stop the process (the process will exist, but will not be scheduled), to terminate the process with a memory image saved to disk (called *core dumped*) and to ignore the signal. In the above examples, the *default signal handler* performed the actions like printing '*Segmentation fault (core dumped)*' (as a result of the *SIGSEGV* signal—memory fault); termination of the process by the *SIGKILL* signal (denoted by the *kill* parameter -9); or termination of the process by the *SIGINT* signal (*CTRL-C*).

So to write your own signal handlers, simply register your own function for a signal; the default signal handler is then no longer invoked, but your function is called when the specified signal is received by the process. Whenever a signal is delivered to the process, the code flow jumps to the signal handler, and after it executes, returns to the code where it was interrupted. For example, if you modify your earlier infinite-loop example to add a signal handler for *CTRL-C*, the resulting code is in the file *inctrl.c*. Compile and run it, and press *CTRL-C*. You will see that the process is not terminated, but prints the following output:

```
bash-3.00$ gcc inctrl.c
bash-3.00$ ./a.out
I have received 2
I have received 2
Quit (core dumped)
bash-3.00$
```

What I did was press *CTRL-C* twice, and then *CTRL-*. The *SIGINT* signal handler prevents *CTRL-C* from being useful—after each execution, control again returns to the *while* loop. The third time, I pressed *CTRL-* and the process was terminated with a core dump, since this is the default action for *CTRL-* and we have not put in a custom handler for that signal. You can use the *gdb* debugger to view the core file; you should find something like, '*Core was generated by ./a.out. Program terminated with signal 3, Quit.*' This means the program terminated due to Signal 3, which on my system is *SIGQUIT*, generated by my pressing *CTRL-*.

For another example, look at *chldsig.c* in the code archive. Compile and run it in the background, like *./a.out&* and you will see that the line '*Received death of Child Signal*' is printed 10 times. The *SIGCHLD* signal is delivered to a parent process on the death of a child process. On receiving *SIGCHLD*, your signal handler in the parent executes.

Please note that not all signals may be handled, like *SIGKILL*, *SIGSTOP*, etc. You may even ignore or block a signal. Improper handling can even cause race conditions in your code. Once you get familiar with the signal interface, you would probably like to use the *sigaction* function instead of *signal*—it gives you finer control.

In this article, we have learnt the basic concepts of process, loading programs, and signal handling. In the next article, we will look at Inter-Process Communication techniques like pipes, FIFO, message queues, etc. I hope you found this useful as an introduction, and will explore the topic a lot more. 

References

- [1] Bach, Maurice J: 'The Design of the Unix Operating System', PHI
- [2] Stevens, W Richard: 'Unix Network Programming: Interprocess Communication', Vol-2, 2nd Edition, Pearson Education
- [3] Stevens, W Richard and Rago, Stephen A: 'Advanced Programming in the Unix Environment', 2nd Edition, Pearson Education
- [4] Linux Manual Pages
- [5] You can download the source code of this article from http://www.linuxforu.com/article_source_code/july12/system_programming.zip

Acknowledgement

I would like to thank Tanmoy Bandyopadhyay and Vikas Nagpal for their help in reviewing this article.

By: Swati Mukhopadhyay

The author is having more than 12 years of experience in academics and corporate training. Her interest areas are Digital logic, Computer architecture, Computer Networking, Data Structures, Linux and programming languages like C, C++. For any queries or feedback, she can be reached at swati.mukherjee@gmail.com or <http://swati.mukhopadhyay.wordpress.com/>.